World Models

Learning to act inside a learned simulation

A concise chapter with a complete, tested PyTorch implementation • October 2026

What you will learn

- What a *world model* is, and why an agent that can imagine the future can learn with far less real experience.
- The three building blocks: a model that *compresses* what the agent sees, a model that *predicts* what happens next, and a way to *choose actions* using those predictions.
- How to build, train and test a complete world-model agent in PyTorch, line by line. The agent learns to play a small game entirely inside its own "dream" and then plays the real game perfectly.
- How to recognise the main failure mode (an agent that *cheats* its world model), and how today's systems (Dreamer, MuZero, TD-MPC2, Genie, V-JEPA 2) build on the same ideas.

Assumed background. Python and basic PyTorch; neural networks, including CNNs and RNNs/LSTMs; basic probability (Gaussian distributions, likelihood). Reinforcement learning (RL) is helpful but not required: Section 2 gives what you need.

Code. Everything in Section 5 is in three files that come with this chapter: `world_model.py` (the complete system, printed in full in Section 5), `dream_experiments.py` and `make_figures.py` (printed in the appendix). All numbers and figures in this chapter come from actually running these files.

Contents

1 What is a world model? 1

2 Just enough reinforcement learning 2

3 Anatomy of a world model 3

4 The classic recipe: V–M–C (Ha and Schmidhuber, 2018) 5

5 Build it: a complete world model in PyTorch 5

6 From this toy to today's world models 18

7 Practical lessons 20

8 Exercises 20

9 Where to learn more 21

A Appendix: the two helper scripts 23

1 What is a world model?

Before you step onto a road you picture the approaching car, guess where it will be in two seconds, and decide whether to wait. You did not have to be hit by a car to learn this; you used a model of the world in your head. In 1943 the psychologist Kenneth Craik suggested that the

mind carries such a "small-scale model" of external reality and uses it to try out alternatives before acting [5].

Definition

A **world model** is a learned model of how an environment changes, especially in response to the agent's own actions. Given what the agent has seen so far and an action it might take, the world model predicts what happens next: the next observation (usually in a compressed form), and often the reward and whether the episode ends:

$$\underbrace{x_{0:t},\ a_{0:t}}_{\text{what I saw, what I do}} \longmapsto\ p(x_{t+1},\, r_t,\, d_t \mid x_{0:t},\, a_{0:t}). \tag{1}$$

Why is this useful? Once an agent can predict the consequences of its actions, it can:

1. **Practise in imagination.** Real experience is slow, expensive or dangerous (robots break, cars crash). Imagined experience is cheap and can be generated thousands of times in parallel. This makes model-based agents much more *sample efficient*.
2. **Plan.** Like a chess player, the agent can try several action sequences "in its head" and pick the best one.
3. **Learn better representations.** To predict the future, a model must capture what matters: positions, speeds, which objects exist even when hidden.
4. **Act as a simulator.** A good world model can replace a hand-written simulator or game engine, for example to generate training worlds for other agents.

The catch. A learned model is never perfect. An agent trained inside an imperfect model may learn to exploit the model's mistakes, behaviour that looks brilliant in imagination and fails in reality. You will see this happen in Section 5.10, and much of the craft of world models is about managing this gap.

A note on the term. "World model" is used in several related ways: (i) in RL and control, an action-conditioned predictive model used for decision making (the focus of this chapter); (ii) large generative "world simulators" that produce video in response to actions or prompts, such as Genie and Cosmos; (iii) predictive representations that capture how the world behaves without generating pixels, such as JEPA; and (iv) the implicit knowledge about the world inside large language models. A recent survey organises the field around two functions, understanding the present and predicting the future [9].

2 Just enough reinforcement learning

An agent interacts with an environment in time steps $t = 0, 1, 2, \ldots$ It receives an *observation* x_t (for us, a 64×64 colour image), chooses an *action* a_t, and the environment returns a *reward* r_t and the next observation x_{t+1}. The goal is to maximise the *return*, the sum of rewards over an episode, $R = \sum_t r_t$. A rule for choosing actions is called a *policy*, $\pi(a_t \mid x_{0:t})$.

Two facts matter for world models. First, one observation is often not the full state of the world: from a single frame you cannot see how fast a ball is moving, so the agent needs *memory*. Second, there are two broad ways to learn a policy (Table 1).

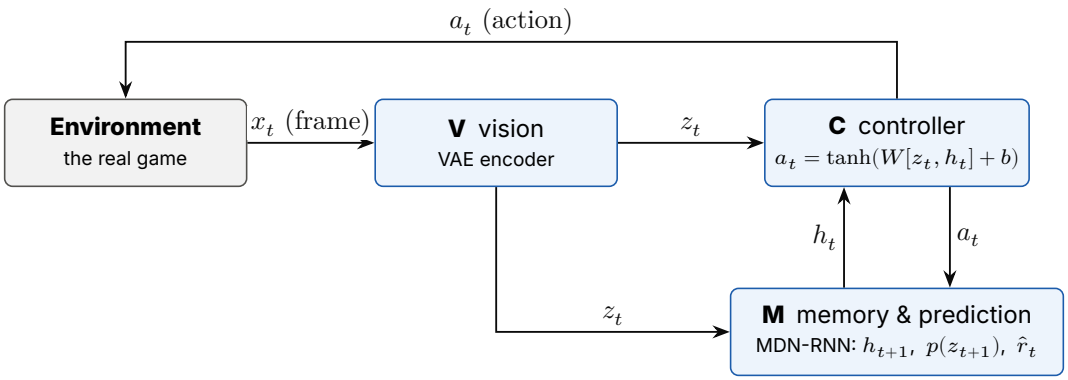


Figure 1. The V–M–C agent [13]. V compresses each frame x_t into a short code z_t. M keeps a memory h_t, predicts the distribution of the next code and the reward, and is updated with (z_t, a_t) after every step. C chooses the action from $[z_t, h_t]$. In a *dream*, the environment and V's encoder are switched off: a code sampled from M is fed back as the next observation.

Table 1. Model-free versus model-based reinforcement learning.

	Model-free RL	**Model-based RL (world models)**
Learns	a policy and/or value function directly from trial and error	a model of the environment, then uses it to train a policy or to plan
Real data needed	a lot	much less: imagined experience is free
Main risk	slow and data-hungry	errors in the model get exploited
Examples	DQN, PPO, SAC	Dyna, PlaNet, Dreamer, MuZero, TD-MPC2

The classic blueprint for model-based learning is *Dyna* [32]: learn a model from real experience, then use it to generate extra, simulated experience for learning. Chapter 8 of Sutton and Barto's textbook is the standard introduction [33]. World models are the deep-learning version of this idea, where the observations are images and the model is a neural network.

3 Anatomy of a world model

We use this notation throughout: x_t is the observation (an image), z_t a short latent vector that summarises x_t, h_t the memory (hidden state of a recurrent network), a_t the action, r_t the reward, and d_t an "episode ended" flag. Figure 1 shows how the parts fit together.

3.1 Perception: compress what you see (V)

A 64×64 image with three colour channels is 12,288 numbers, most of them unimportant (a black background, say). Predicting the future directly in pixels is possible (Section 6) but expensive, so the classic approach first learns a compact code. A *variational autoencoder* (VAE) [24] has an *encoder* that maps an image x to a Gaussian distribution over a latent vector, $q(z \mid x) = \mathcal{N}(\mu(x), \operatorname{diag}\sigma^2(x))$, and a *decoder* that maps z back to an image. It is trained by minimising

$$\mathcal{L}_{\mathrm{V}} = \underbrace{\mathbb{E}_{z\sim q(z|x)}\big[-\log p(x \mid z)\big]}_{\text{reconstruction}} + \underbrace{\mathrm{KL}\big(q(z \mid x)\,\|\,\mathcal{N}(0, I)\big)}_{\text{keep codes near a standard Gaussian}} . \tag{2}$$

The first term asks the decoded image to match the input. The second keeps the codes close to a standard Gaussian, which makes the latent space smooth: nearby codes decode to similar images. This matters later, because M will *generate* codes and the decoder must turn them into sensible images. To back-propagate through the random sample we use the *reparameterisation trick*: $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$.

3.2 Prediction: learn how the world changes (M)

Next we learn how the codes change when the agent acts. A recurrent network with hidden state h_t, here an LSTM [23], reads one step at a time:

$$h_{t+1} = \mathrm{LSTM}\,(h_t,\,[z_t, a_t]), \qquad p(z_{t+1} \mid z_t, a_t, h_t) \text{ is computed from } h_{t+1}. \tag{3}$$

The hidden state is the agent's memory: it can encode speeds, directions and anything else a single frame does not show.

Why predict a distribution instead of a single value? Because the future is often uncertain. In our game, when a ball lands, the next ball appears at a random position. A network trained with squared error would predict the *average* position, a ball in the middle that never actually occurs. A *mixture density network* (MDN) [3] instead outputs a mixture of K Gaussians. Following Ha and Schmidhuber, we use an independent mixture for each latent dimension i:

$$p(z_{t+1,i}) \;=\; \sum_{k=1}^{K} \pi_{k,i}\;\mathcal{N}(z_{t+1,i};\;\mu_{k,i},\;\sigma^2_{k,i}), \qquad \sum_k \pi_{k,i} = 1. \tag{4}$$

The network outputs the mixture weights π (through a softmax), the means μ and the log standard deviations $\log\sigma$, and is trained to minimise the negative log-likelihood (NLL) of the true next code. An RNN with an MDN output is called an *MDN-RNN*; it was used earlier to generate handwriting [11] and sketches [12].

Temperature. When sampling from M we can add a temperature τ, as in SketchRNN: the mixture weights become $\pi_k \propto \exp(\log \pi_k/\tau)$ and each standard deviation becomes $\sigma\sqrt{\tau}$. With $\tau > 1$ the imagined future is more random; as $\tau \to 0$ it becomes almost deterministic.

3.3 Rewards and episode ends

Small extra output layers ("heads") on h_{t+1} predict the reward $\hat{r}_t$ and, if needed, the probability that the episode ends. With these heads, M is a complete simulator: start from the code of a real frame, choose an action, sample the next code, read off the reward, and repeat. Running an agent inside this simulator is called *dreaming* or *imagination*.

3.4 Using the model to act

There are two main ways to turn predictions into behaviour:

- **Learn a policy in imagination.** Run the policy inside the model, measure the imagined return, improve the policy, and never touch the real environment while doing so. Ha and Schmidhuber evolved a tiny linear controller with an evolution strategy [13]; Dreamer trains a neural actor and critic on imagined trajectories [16]. Acting is then cheap: one forward pass per step.
- **Plan at decision time.** At every step, imagine many candidate action sequences, score them with the model, execute the first action of the best one, and plan again at the next step. This is *model-predictive control* (MPC). Examples are the cross-entropy method in PlaNet [15] and V-JEPA 2-AC [2], MPPI in TD-MPC2 [22], and Monte-Carlo tree search in MuZero [31]. Planning needs no policy training but costs much more computation per step.

3.5 The overall training loop

Whatever the design, training alternates between (1) collecting experience in the real environment, (2) fitting the world model to it, (3) improving behaviour with the model, and (4) collecting new experience with the improved agent. Step 4 matters when random actions never reach the interesting parts of the world: in a hard game the agent must first get better before it can see, and learn to model, the later levels [13]. In the simple game of this chapter, one round with random data is enough.

4 The classic recipe: V–M–C (Ha and Schmidhuber, 2018)

Ha and Schmidhuber's *World Models* [13, 14] turned these ideas into a simple, influential recipe with three separately trained parts (Figure 1):

- **V**, a convolutional VAE that compresses each 64×64 RGB frame into z (32 numbers for car racing, 64 for Doom);
- **M**, an MDN-RNN with an LSTM of 256 or 512 units and 5 Gaussians per latent dimension;
- **C**, a linear controller $a_t = W_c[z_t, h_t] + b_c$ (867 parameters for car racing), trained with the evolution strategy CMA-ES [20] using a population of 64 candidates, each tested on 16 rollouts.

Almost all parameters (millions) sit in the world model, which is trained with ordinary backpropagation, while the controller is so small that its parameters can be searched by evolution, which only needs the total reward of each rollout.

Two results made the paper famous. In *CarRacing-v0*, learning to drive from pixels, the agent averaged 906 ± 21 points over 100 random tracks, the first reported solution of the task (900 counts as solved). Giving C only z_t, without M's memory h_t, dropped the score to 632 ± 251: prediction and memory matter. In *VizDoom: Take Cover*, the controller was trained *entirely inside M's dream* and then deployed in the real game, where it survived about 1,100 time steps on average (750 counts as solved). The paper also exposed the danger of dreaming: at a low temperature ($\tau = 0.1$) the dreamed monsters stopped shooting fireballs, and the controller scored 2,086 in the dream but only 193 in reality. At $\tau = 1.15$ it scored 918 in the dream and 1,092 in reality. We now rebuild this recipe end to end.

5 Build it: a complete world model in PyTorch

5.1 The task and the plan

We use a tiny game called *Catch* (top row of Figure 3). A ball falls from the top of a 64×64 image, and the agent moves a paddle along the bottom with a continuous action $a \in [-1, 1]$. It earns $+1$ for each ball it catches. An episode has five balls (80 steps), so the best possible score is 5. When a ball lands, the next one appears at a random position. The agent sees only pixels.

The game is small enough that the whole pipeline runs in about 12 minutes on a 2-core cloud CPU, and usually faster on a machine with more CPU cores or a GPU. Yet it contains everything a real system must handle: images as input, a random event (where the next ball appears), a sparse reward (only when a ball lands), and the need to start moving long before the ball arrives. Figure 2 shows the plan and Table 2 the three models.

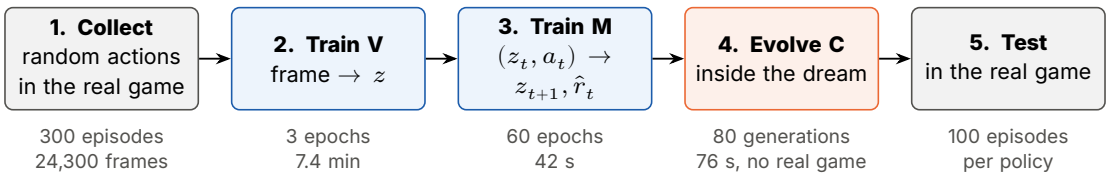


Figure 2. The pipeline run by `world_model.py`, with the time each stage took on a 2-core CPU. Grey stages use the real game, blue stages are ordinary gradient-based training on the recorded data, and the orange stage happens entirely in the dream.

Table 2. The three models. V has exactly the layer sizes of the paper's car-racing VAE, and therefore exactly the parameter count reported there.

	Implementation	Parameters
V	convolutional VAE, $64 \times 64 \times 3$ image $\to z \in \mathbb{R}^{32}$, 4 conv + 4 transposed-conv layers	4,348,547
M	LSTM with 256 units, MDN with 5 Gaussians per latent dimension, and a reward head	421,601
C	linear map from $[z_t, h_t]$ (288 numbers) to one action, followed by tanh	289

How to run it

```
pip install torch numpy cma        # cma provides CMA-ES
python world_model.py              # trains V, M and C, tests them, saves runs/
python dream_experiments.py        # optional: the experiments of Section 5.10
python make_figures.py             # optional: Figures 3 and 4
```

Listings 1 to 7 below are the complete file `world_model.py`, in order; only the blank lines between listings are left out, and the line numbers match the file. Read them from top to bottom: each listing builds on the previous ones.

5.2 Settings

All hyperparameters live in one `Config` dataclass. The values are deliberately small so that everything runs on a CPU. The GPU is used automatically if PyTorch finds one.

Listing 1 Imports and settings

```
"""
world_model.py: a complete, minimal World Models agent in PyTorch.

It follows the V-M-C design of Ha & Schmidhuber (2018):
  V  a convolutional VAE that compresses each 64x64 RGB frame into a vector z (32 numbers)
  M  an MDN-RNN (LSTM + mixture density output) that predicts the next z and the reward
  C  a tiny linear controller, trained with CMA-ES entirely inside M's "dream"

main() runs the whole pipeline:
  1. collect episodes with a random policy in the real environment
  2. train V on the frames, then encode every frame into (mu, logvar)
  3. train M on the latent sequences
  4. evolve C inside the dream (the real environment is not used in this step)
  5. test C in the real environment and compare it with baselines, including
     a planner that reuses the same world model (model-predictive control with CEM)

Install:  pip install torch numpy cma
Run:      python world_model.py
"""
import json
import math
import os
import time
from dataclasses import asdict, dataclass

import cma  # CMA-ES optimiser
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```python


# ---------------------------------------------------------------------------
# 0. Settings
# ---------------------------------------------------------------------------
@dataclass
class Config:
    seed: int = 0
    out_dir: str = "runs"
    # data
    n_episodes: int = 300        # random-policy episodes (300 x 80 = 24,000 steps)
    # V: convolutional VAE
    z_dim: int = 32
    vae_epochs: int = 3
    vae_batch: int = 32
    vae_lr: float = 1e-3
    kl_tolerance: float = 0.5    # "free bits": KL below 0.5 nats per dim is not penalised
    # M: MDN-RNN
    hidden: int = 256
    n_mix: int = 5
    rnn_epochs: int = 60
    rnn_batch: int = 16
    rnn_lr: float = 1e-3
    # C: controller evolved inside the dream
    temperature: float = 1.15    # > 1 makes the dream noisier, so it is harder to exploit
    popsize: int = 32
    dream_rollouts: int = 16     # dreams per candidate; fitness = average dream return
    generations: int = 80
    sigma0: float = 0.1
    # evaluation and planning
    eval_episodes: int = 100
    plan_episodes: int = 20
    plan_horizon: int = 16       # imagine 16 steps ahead (one full ball drop)
    plan_candidates: int = 100
    plan_elites: int = 10
    plan_iters: int = 2
```

5.3 The environment and the data

Listing 2 defines the game and collects the training data. Three details are worth noting:

- `CatchEnv.step` returns only an image, a reward and a "done" flag, just like a Gym-style environment. The agent never sees the ball's coordinates.
- The random policy repeats each random action for 1–8 steps. Pure per-step noise would make the paddle jitter in place; holding actions moves it across the screen, so the data covers more situations, including some catches: 1.72 per episode on average in our data.
- `collect_episodes` stores 81 frames, 80 actions and 80 rewards per episode. Reward r_t is the reward received *after* taking action a_t in frame x_t.

Listing 2 The *Catch* game, a random policy and data collection (step 1)

```python
# ---------------------------------------------------------------------------
# 1. The environment: a tiny "Catch" game, observed only as 64x64 RGB frames
# ---------------------------------------------------------------------------
class CatchEnv:
    """A ball falls from the top; move the paddle (action in [-1, 1]) to catch it.
    Reward is +1 per catch. Each episode has 5 balls, i.e. 80 steps."""
    SIZE = 64
    PADDLE_W, PADDLE_H, PADDLE_Y, PADDLE_SPEED = 16, 4, 58, 4.0
    BALL_R, BALL_Y0, BALL_SPEED = 4, 6.0, 3.0
```

```
    PADDLE_RGB, BALL_RGB = (60, 160, 255), (255, 140, 40)    # blue paddle, orange ball
    N_BALLS = 5
    STEPS_PER_BALL = math.ceil((PADDLE_Y - BALL_R - BALL_Y0) / BALL_SPEED)  # 16
    EPISODE_LEN = N_BALLS * STEPS_PER_BALL                                   # 80

    def __init__(self, seed=None):
        self.rng = np.random.default_rng(seed)
        self.ys, self.xs = np.mgrid[0:self.SIZE, 0:self.SIZE]

    def reset(self):
        half = self.PADDLE_W / 2
        self.paddle_x = self.rng.uniform(half, self.SIZE - half)
        self.balls_left = self.N_BALLS
        self._new_ball()
        return self.render()

    def _new_ball(self):
        self.ball_x = self.rng.uniform(self.BALL_R, self.SIZE - self.BALL_R)
        self.ball_y = self.BALL_Y0

    def step(self, action):
        half = self.PADDLE_W / 2
        move = float(np.clip(action, -1, 1)) * self.PADDLE_SPEED
        self.paddle_x = float(np.clip(self.paddle_x + move, half, self.SIZE - half))
        self.ball_y += self.BALL_SPEED
        reward = 0.0
        if self.ball_y >= self.PADDLE_Y - self.BALL_R:     # ball reached the paddle row
            reward = float(abs(self.ball_x - self.paddle_x) <= half + self.BALL_R / 2)
            self.balls_left -= 1
            self._new_ball()                               # the next ball: random x
        return self.render(), reward, self.balls_left == 0

    def render(self):
        img = np.zeros((self.SIZE, self.SIZE, 3), dtype=np.uint8)
        x0, y0 = int(round(self.paddle_x - self.PADDLE_W / 2)), self.PADDLE_Y
        img[y0:y0 + self.PADDLE_H, x0:x0 + self.PADDLE_W] = self.PADDLE_RGB
        dist2 = (self.xs - self.ball_x) ** 2 + (self.ys - self.ball_y) ** 2
        img[dist2 <= self.BALL_R ** 2] = self.BALL_RGB
        return img


class RandomPolicy:
    """Random actions, each held for 1-8 steps (explores more widely than pure noise)."""
    def __init__(self, rng):
        self.rng, self.action, self.steps_left = rng, 0.0, 0

    def __call__(self, obs=None):
        if self.steps_left == 0:
            self.action = self.rng.uniform(-1, 1)
            self.steps_left = int(self.rng.integers(1, 9))
        self.steps_left -= 1
        return self.action


def collect_episodes(n, seed):
    """Step 1: play n random-policy episodes; record frames, actions and rewards."""
    env, rng, T = CatchEnv(seed), np.random.default_rng(seed + 1), CatchEnv.EPISODE_LEN
    obs = np.zeros((n, T + 1, 64, 64, 3), np.uint8)   # frames  x_0 ... x_T
    act = np.zeros((n, T, 1), np.float32)             # actions a_0 ... a_{T-1}
    rew = np.zeros((n, T), np.float32)                # rewards r_0 ... r_{T-1}
    for i in range(n):
```

```
        policy = RandomPolicy(rng)
        obs[i, 0] = env.reset()
        for t in range(T):
            act[i, t, 0] = policy()
            obs[i, t + 1], rew[i, t], _ = env.step(act[i, t, 0])
    return obs, act, rew


def to_tensor(frames):
    """uint8 frames [B, 64, 64, 3] -> float tensor [B, 3, 64, 64], values in [0, 1]."""
    return torch.from_numpy(frames).to(DEVICE).permute(0, 3, 1, 2).float() / 255.0
```

5.4 V: the variational autoencoder

Listing 3 implements Equation (2). The encoder halves the image size four times ($64 \to 31 \to 14 \to 6 \to 2$) and outputs μ and $\log \sigma^2$. The decoder mirrors it. Two choices differ slightly from the paper:

- **Reconstruction loss.** The paper used squared error (L2). We use binary cross-entropy, a common alternative for pixel values in $[0, 1]$. On our mostly black frames, the L2 version stayed stuck for several hundred steps producing an empty black image, the cheapest way to reduce its error. Cross-entropy penalises confident mistakes on the few bright pixels much more, and learned the ball and paddle sooner. Because the loss works on the decoder's *logits*, the final sigmoid is applied in `decode()`.
- **"Free bits".** Like the original code, the KL term of an image is not penalised while it is below 16 nats (0.5 nats for each of the 32 latent dimensions). This stops the encoder from throwing information away too eagerly early in training.

After training, `encode_episodes` stores $(\mu, \log \sigma^2)$ for every frame. From now on M works only with these 32-number codes, never with pixels.

Listing 3 V: convolutional VAE, its loss and training (step 2)

```
# ------------------------------------------------------------------------
# 2. V: the convolutional VAE (same layer sizes as in the World Models paper)
# ------------------------------------------------------------------------
class ConvVAE(nn.Module):
    def __init__(self, z_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, 4, stride=2), nn.ReLU(),       # 64x64 -> 31x31
            nn.Conv2d(32, 64, 4, stride=2), nn.ReLU(),      # -> 14x14
            nn.Conv2d(64, 128, 4, stride=2), nn.ReLU(),     # -> 6x6
            nn.Conv2d(128, 256, 4, stride=2), nn.ReLU(),    # -> 2x2
            nn.Flatten())                                   # -> 1024
        self.fc_mu = nn.Linear(1024, z_dim)
        self.fc_logvar = nn.Linear(1024, z_dim)
        self.fc_dec = nn.Linear(z_dim, 1024)
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(1024, 128, 5, stride=2), nn.ReLU(),  # 1x1 -> 5x5
            nn.ConvTranspose2d(128, 64, 5, stride=2), nn.ReLU(),    # -> 13x13
            nn.ConvTranspose2d(64, 32, 6, stride=2), nn.ReLU(),     # -> 30x30
            nn.ConvTranspose2d(32, 3, 6, stride=2))                 # -> 64x64 (logits)

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def decode_logits(self, z):
```

```
        return self.decoder(self.fc_dec(z).view(-1, 1024, 1, 1))

    def decode(self, z):
        return torch.sigmoid(self.decode_logits(z))        # image with values in [0, 1]

    def forward(self, x):
        mu, logvar = self.encode(x)
        # reparameterisation trick: z = mu + sigma * eps with eps ~ N(0, I)
        z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
        return self.decode_logits(z), mu, logvar


def vae_loss(x, logits, mu, logvar, kl_tolerance):
    """Reconstruction (binary cross-entropy per pixel) + KL divergence, both summed per
    image and averaged over the batch. As in the original code, the KL term is floored
    at kl_tolerance * z_dim ("free bits")."""
    bce = F.binary_cross_entropy_with_logits(logits, x, reduction="none")
    rec = bce.sum(dim=(1, 2, 3)).mean()
    kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum(dim=1)
    kl = torch.clamp(kl, min=kl_tolerance * mu.shape[1]).mean()
    return rec + kl, rec, kl


def train_vae(frames, cfg):
    """Step 2a: learn to compress single frames (unsupervised: no actions, no rewards)."""
    vae = ConvVAE(cfg.z_dim).to(DEVICE)
    opt = torch.optim.Adam(vae.parameters(), lr=cfg.vae_lr)
    log, steps_per_epoch = [], math.ceil(len(frames) / cfg.vae_batch)
    for epoch in range(cfg.vae_epochs):
        perm = np.random.permutation(len(frames))
        for i in range(0, len(frames), cfg.vae_batch):
            x = to_tensor(frames[perm[i:i + cfg.vae_batch]])
            logits, mu, logvar = vae(x)
            loss, rec, kl = vae_loss(x, logits, mu, logvar, cfg.kl_tolerance)
            opt.zero_grad()
            loss.backward()
            opt.step()
            log.append([rec.item(), kl.item()])              # losses of every step
        rec_avg, kl_avg = np.mean(log[-steps_per_epoch:], axis=0)
        print(f"   VAE epoch {epoch + 1}/{cfg.vae_epochs}: "
              f"reconstruction {rec_avg:6.1f}  KL {kl_avg:5.1f}")
    return vae, log


@torch.no_grad()
def encode_episodes(vae, obs, batch=1000):
    """Step 2b: turn every frame into (mu, logvar). M trains on these, never on pixels."""
    flat = obs.reshape(-1, 64, 64, 3)
    mus, logvars = [], []
    for i in range(0, len(flat), batch):
        mu, logvar = vae.encode(to_tensor(flat[i:i + batch]))
        mus.append(mu.cpu())
        logvars.append(logvar.cpu())
    shape = obs.shape[:2] + (-1,)                       # [episodes, T + 1, z_dim]
    return torch.cat(mus).view(shape), torch.cat(logvars).view(shape)
```

5.5 M: the MDN-RNN

Listing 4 implements Equations (3) and (4). For a batch of sequences, `forward` returns, at every step t, the mixture over z_{t+1} (three tensors of shape `[B, T, 32, 5]`), the predicted reward $\hat{r}_t$, and the LSTM state. Points to notice:

- **Log-sum-exp.** `mdn_nll` computes $\log \sum_k \pi_k \mathcal{N}(\cdot)$ as `logsumexp(log pi + log N)`, which avoids numerical underflow.
- **Teacher forcing.** During training, M always receives the *true* code z_t as input (inputs $z_0, \dots, z_{T-1}$; targets $z_1, \dots, z_T$), not its own prediction.
- **Fresh samples.** As in the paper's appendix, each batch uses a new sample $z \sim \mathcal{N}(\mu, \sigma^2)$ rather than always the mean. This acts as data augmentation and makes M robust to slightly noisy codes.
- **Sampling.** `sample_mdn` first picks one mixture component per dimension, then samples from that Gaussian, applying the temperature τ. Gradient clipping at 1.0 and the 5 mixtures match the original code.

After 60 epochs (42 s), M predicts the reward correctly (using a threshold of 0.5) at 98.7% of the 150 ball landings in 30 new random episodes, and never predicts a reward at the other steps.

Listing 4 M: the MDN-RNN, its loss, sampling and training (step 3)

```
# ------------------------------------------------------------------------
# 3. M: the MDN-RNN, which models P(z_{t+1} | a_t, z_t, h_t) and predicts r_t
# ------------------------------------------------------------------------
class MDNRNN(nn.Module):
    def __init__(self, z_dim=32, action_dim=1, hidden=256, n_mix=5):
        super().__init__()
        self.z_dim, self.n_mix, self.hidden = z_dim, n_mix, hidden
        self.lstm = nn.LSTM(z_dim + action_dim, hidden, batch_first=True)
        # for every z-dimension and every mixture component: log pi, mu and log sigma
        self.mdn = nn.Linear(hidden, 3 * n_mix * z_dim)
        self.reward = nn.Linear(hidden, 1)

    def forward(self, z, a, state=None):
        """z: [B, T, z_dim], a: [B, T, action_dim]. For every step t this returns the
        mixture over z_{t+1}, the predicted reward r_t and the LSTM state (h, c) after
        seeing (z_t, a_t)."""
        out, state = self.lstm(torch.cat([z, a], dim=-1), state)
        B, T, _ = out.shape
        p = self.mdn(out).view(B, T, self.z_dim, 3 * self.n_mix)
        logpi, mu, logsigma = p.split(self.n_mix, dim=-1)  # each [B, T, z_dim, n_mix]
        logpi = F.log_softmax(logpi, dim=-1)               # mixture weights sum to 1
        return logpi, mu, logsigma, self.reward(out).squeeze(-1), state


def mdn_nll(logpi, mu, logsigma, target):
    """Negative log-likelihood of target [B, T, z_dim] under a factored mixture of
    Gaussians, averaged over batch, time and z-dimensions (as in the original code)."""
    target = target.unsqueeze(-1)
    # log N(target; mu, sigma^2) for every component
    log_gauss = (-0.5 * ((target - mu) / logsigma.exp()) ** 2
                 - logsigma - 0.5 * math.log(2 * math.pi))
    return -torch.logsumexp(logpi + log_gauss, dim=-1).mean()


def sample_mdn(logpi, mu, logsigma, tau=1.0):
    """Sample z_{t+1}: pick a component per dimension, then sample from that Gaussian.
    Temperature tau > 1 flattens the mixture weights and widens each Gaussian
    (as in SketchRNN)."""
    k = torch.distributions.Categorical(logits=logpi / tau).sample().unsqueeze(-1)
    mu_k = mu.gather(-1, k).squeeze(-1)
    sigma_k = logsigma.gather(-1, k).squeeze(-1).exp()
    return mu_k + sigma_k * math.sqrt(tau) * torch.randn_like(mu_k)

```

```
280
281 def train_mdnrnn(mu, logvar, act, rew, cfg):
282     """Step 3: teach M to predict the next latent vector and the reward."""
283     rnn = MDNRNN(cfg.z_dim, act.shape[-1], cfg.hidden, cfg.n_mix).to(DEVICE)
284     opt = torch.optim.Adam(rnn.parameters(), lr=cfg.rnn_lr)
285     log, n = [], len(mu)
286     for epoch in range(cfg.rnn_epochs):
287         perm, sums = torch.randperm(n), np.zeros(2)
288         for i in range(0, n, cfg.rnn_batch):
289             idx = perm[i:i + cfg.rnn_batch]
290             m, lv = mu[idx].to(DEVICE), logvar[idx].to(DEVICE)
291             # a fresh sample z ~ N(mu, sigma^2) for every batch (data augmentation)
292             z = m + torch.exp(0.5 * lv) * torch.randn_like(m)
293             a, r = act[idx].to(DEVICE), rew[idx].to(DEVICE)
294             # teacher forcing: inputs are the true z_0 ... z_{T-1}, targets z_1 ... z_T
295             logpi, mmu, logsigma, r_pred, _ = rnn(z[:, :-1], a)
296             loss_z = mdn_nll(logpi, mmu, logsigma, z[:, 1:])
297             loss_r = F.mse_loss(r_pred, r)
298             opt.zero_grad()
299             (loss_z + loss_r).backward()
300             nn.utils.clip_grad_norm_(rnn.parameters(), 1.0)
301             opt.step()
302             sums += [loss_z.item() * len(idx), loss_r.item() * len(idx)]
303         log.append((sums / n).tolist())
304         if (epoch + 1) % 10 == 0:
305             print(f"   MDN-RNN epoch {epoch + 1:3d}/{cfg.rnn_epochs}: "
306                   f"z NLL {log[-1][0]:6.3f}  reward MSE {log[-1][1]:.4f}")
307     return rnn, log
```

5.6 C: learning inside the dream

Listing 5 is the heart of the chapter. `dream_returns` runs the dream: it starts from codes of real first frames, and then only M is used. At each step the controllers choose actions, M updates its memory, predicts the reward and *samples* the next code, which becomes the next "observation". No pixels are produced, so V's decoder is not needed at all. Every candidate controller is tested on 16 dreams, and all $32 \times 16 = 512$ dreams of a generation run in parallel as one batch.

CMA-ES in three sentences. CMA-ES [20, 21] keeps a multivariate Gaussian over the 289 controller parameters. In each generation it samples a population of candidates (`ask`), receives their scores (`tell`), moves its mean towards the better candidates and adapts its covariance, that is, the size and shape of its search cloud. It needs only the total reward of each rollout, so a sparse, noisy reward is no problem; we return the mean of the final search distribution.

Listing 5 C: a linear controller evolved with CMA-ES inside the dream (step 4)

```
310 # ------------------------------------------------------------------------
311 # 4. C: a linear controller, evolved with CMA-ES inside the dream
312 # ------------------------------------------------------------------------
313 def controller_action(params, z, h):
314     """a = tanh(W [z, h] + b), for many controllers at once.
315     params: [P, z_dim + hidden + 1], one row per controller;
316     z: [P, R, z_dim] and h: [P, R, hidden], for R rollouts per controller."""
317     W, b = params[:, :-1], params[:, -1:]
318     x = torch.cat([z, h], dim=-1)
319     return torch.tanh(torch.einsum("prf,pf->pr", x, W) + b).unsqueeze(-1)  # [P, R, 1]
320
321
322 @torch.no_grad()
323 def dream_returns(rnn, params, z0_mu, z0_logvar, cfg):
324     """Run P controllers x R dreams in parallel inside M and return the average predicted
```

```
    return of each controller. No pixels and no real environment are involved."""
    P, R = len(params), cfg.dream_rollouts
    params = torch.as_tensor(np.asarray(params), dtype=torch.float32, device=DEVICE)
    idx = torch.randint(len(z0_mu), (R,)).repeat(P)    # same R start frames for all P
    m, lv = z0_mu[idx].to(DEVICE), z0_logvar[idx].to(DEVICE)
    z = m + torch.exp(0.5 * lv) * torch.randn_like(m)       # z_0 from real first frames
    h, state = torch.zeros(P * R, cfg.hidden, device=DEVICE), None
    total = torch.zeros(P * R, device=DEVICE)
    for t in range(CatchEnv.EPISODE_LEN):
        a = controller_action(params, z.view(P, R, -1), h.view(P, R, -1)).view(P * R, 1)
        logpi, mu, logsigma, r, state = rnn(z[:, None], a[:, None], state)
        total += r[:, 0]                                    # predicted reward r_t
        # the dream: z_{t+1} is sampled from M (at temperature tau) instead of observed
        z = sample_mdn(logpi[:, 0], mu[:, 0], logsigma[:, 0], cfg.temperature)
        h = state[0][0]                                     # h_{t+1}
    return total.view(P, R).mean(dim=1).cpu().numpy()


def train_controller_in_dream(rnn, z0_mu, z0_logvar, cfg):
    """Step 4: CMA-ES proposes controllers, the dream scores them, CMA-ES adapts."""
    n_params = cfg.z_dim + cfg.hidden + 1                   # 32 + 256 + 1 = 289
    options = {"popsize": cfg.popsize, "seed": cfg.seed + 1, "verbose": -9}
    es = cma.CMAEvolutionStrategy(np.zeros(n_params), cfg.sigma0, options)
    log = []
    for gen in range(cfg.generations):
        candidates = es.ask()
        fitness = dream_returns(rnn, candidates, z0_mu, z0_logvar, cfg)
        es.tell(candidates, (-fitness).tolist())            # CMA-ES minimises, so negate
        log.append([float(fitness.mean()), float(fitness.max())])
        if (gen + 1) % 10 == 0:
            print(f"   generation {gen + 1:3d}: "
                  f"dream return mean {log[-1][0]:.2f}  best {log[-1][1]:.2f}")
    mean = torch.tensor(es.result.xfavorite, dtype=torch.float32, device=DEVICE)
    return mean[None], log             # the mean of the final search distribution
```

5.7 Back to reality: acting, planning and testing

Listing 6 puts the agent into the real game. `WorldModelAgent` follows the paper's pseudocode exactly: encode the frame, choose an action from $[z_t, h_t]$, then update M's memory with (z_t, a_t). It uses the mean μ of V's output as z_t.

The same class can also act by **planning**, using `cem_plan`, the cross-entropy method (CEM) [30]. It keeps a Gaussian over sequences of 16 future actions (one full ball drop), samples 100 sequences, imagines each with M, keeps the 10 with the highest imagined return, refits the Gaussian to them, and repeats once more. Only the first action is executed, and planning starts again at the next step. No controller is trained at all: the world model alone decides. The hand-coded policy, which finds the ball and paddle by their colours, gives a reference for perfect play.

Listing 6 The agent in the real game, CEM planning and evaluation (step 5)

```
# ---------------------------------------------------------------------------
# 5. The agent in the real world, and a planning alternative to C
# ---------------------------------------------------------------------------
@torch.no_grad()
def cem_plan(rnn, z, state, cfg):
    """Model-predictive control: imagine many action sequences inside M, keep the best
    ones (cross-entropy method), and return only the first action of the final plan."""
    H, N, E = cfg.plan_horizon, cfg.plan_candidates, cfg.plan_elites
    mean, std = torch.zeros(H, device=DEVICE), torch.ones(H, device=DEVICE)
    for _ in range(cfg.plan_iters):
```

```
        actions = (mean + std * torch.randn(N, H, device=DEVICE)).clamp(-1, 1)   # [N, H]
        s = None if state is None else tuple(x.repeat(1, N, 1) for x in state)
        zz, ret = z.repeat(N, 1), torch.zeros(N, device=DEVICE)
        for k in range(H):
            logpi, mu, logsigma, r, s = rnn(zz[:, None], actions[:, k, None, None], s)
            ret += r[:, 0]
            zz = sample_mdn(logpi[:, 0], mu[:, 0], logsigma[:, 0])
        elite = actions[ret.topk(E).indices]                # the E best imagined futures
        mean, std = elite.mean(dim=0), elite.std(dim=0) + 0.05
    return mean[0].item()


class WorldModelAgent:
    """The full agent, exactly like the paper's rollout() pseudocode:
         z = V(obs);  a = C([z, h]);  obs, r, done = env.step(a);  h = M(z, a, h)
    With plan=True, the action comes from CEM planning inside M instead of from C."""
    def __init__(self, vae, rnn, params=None, cfg=None, plan=False):
        self.vae, self.rnn, self.params, self.cfg, self.plan = vae, rnn, params, cfg, plan

    def reset(self):
        self.state, self.h = None, torch.zeros(1, self.rnn.hidden, device=DEVICE)

    @torch.no_grad()
    def __call__(self, obs):
        z = self.vae.encode(to_tensor(obs[None]))[0]       # z_t = mean of q(z | frame)
        if self.plan:
            a_plan = cem_plan(self.rnn, z, self.state, self.cfg)
            a = torch.tensor([[a_plan]], device=DEVICE)
        else:
            a = controller_action(self.params, z[None], self.h[None])[0]
        *_, self.state = self.rnn(z[:, None], a[:, None], self.state)   # memory update
        self.h = self.state[0][0]
        return a.item()


def handcoded_policy(obs):
    """Baseline: find the orange ball and blue paddle by colour; move toward the ball."""
    ball_x = np.argwhere(obs[..., 0] > 200)[:, 1].mean()
    paddle_x = np.argwhere(obs[..., 2] > 200)[:, 1].mean()
    return float(np.clip((ball_x - paddle_x) / CatchEnv.PADDLE_SPEED, -1, 1))


def evaluate(policy, n_episodes, seed=12345):
    """Balls caught per episode (mean, std; max 5). All policies see the same balls."""
    env, scores = CatchEnv(seed), []
    for _ in range(n_episodes):
        obs, done, total = env.reset(), False, 0.0
        if hasattr(policy, "reset"):
            policy.reset()
        while not done:
            obs, r, done = env.step(policy(obs))
            total += r
        scores.append(total)
    return float(np.mean(scores)), float(np.std(scores))
```

5.8 Putting it all together

Listing 7 The complete pipeline

```
# ----------------------------------------------------------------------------
# 6. Put it all together
# ----------------------------------------------------------------------------
def main(cfg=None):
    cfg = cfg or Config()
    torch.manual_seed(cfg.seed)
    np.random.seed(cfg.seed)
    os.makedirs(cfg.out_dir, exist_ok=True)
    start, timings = time.time(), {}

    print("1) Collecting random-policy episodes")
    obs, act, rew = collect_episodes(cfg.n_episodes, cfg.seed)
    catches = rew.mean() * CatchEnv.EPISODE_LEN
    print(f"   {act.shape[0] * act.shape[1]} steps, {catches:.2f} catches/episode")
    timings["collect"] = time.time() - start

    print("2) Training V (convolutional VAE)")
    vae, vae_log = train_vae(obs.reshape(-1, 64, 64, 3), cfg)
    mu, logvar = encode_episodes(vae, obs)
    timings["vae"] = time.time() - start

    print("3) Training M (MDN-RNN)")
    act, rew = torch.from_numpy(act), torch.from_numpy(rew)
    rnn, rnn_log = train_mdnrnn(mu, logvar, act, rew, cfg)
    timings["rnn"] = time.time() - start

    print("4) Evolving C inside the dream (CMA-ES)")
    params, es_log = train_controller_in_dream(rnn, mu[:, 0], logvar[:, 0], cfg)
    timings["controller"] = time.time() - start
    torch.save(vae.state_dict(), os.path.join(cfg.out_dir, "vae.pt"))
    torch.save(rnn.state_dict(), os.path.join(cfg.out_dir, "mdnrnn.pt"))
    torch.save(params.cpu(), os.path.join(cfg.out_dir, "controller.pt"))

    print("5) Testing in the real environment")
    random_policy = RandomPolicy(np.random.default_rng(1))
    agent = WorldModelAgent(vae, rnn, params)                    # V + M + dream-trained C
    planner = WorldModelAgent(vae, rnn, cfg=cfg, plan=True)      # V + M + CEM planning
    results = {
        "random policy": evaluate(random_policy, cfg.eval_episodes),
        "world model + dream-trained C": evaluate(agent, cfg.eval_episodes),
        "world model + CEM planning": evaluate(planner, cfg.plan_episodes),
        "hand-coded policy": evaluate(handcoded_policy, cfg.eval_episodes),
    }
    for name, (mean, std) in results.items():
        print(f"   {name:30s} {mean:.2f} +/- {std:.2f} balls caught (out of 5)")
    timings["total"] = time.time() - start
    log = {"config": asdict(cfg), "vae": vae_log, "mdnrnn": rnn_log, "cmaes": es_log,
           "results": results, "timings_s": timings}
    with open(os.path.join(cfg.out_dir, "log.json"), "w") as f:
        json.dump(log, f, indent=1)
    print(f"Finished in {timings['total'] / 60:.1f} minutes. "
          f"Models and logs saved to '{cfg.out_dir}/'.")


if __name__ == "__main__":
    main()
```

5.9 Results

This is the complete output of `python world_model.py` on a 2-core cloud CPU (12.4 minutes in total):

```
1) Collecting random-policy episodes
   24000 steps, 1.72 catches/episode
2) Training V (convolutional VAE)
   VAE epoch 1/3: reconstruction  606.8  KL  30.9
   VAE epoch 2/3: reconstruction  186.4  KL  39.3
   VAE epoch 3/3: reconstruction  169.5  KL  32.8
3) Training M (MDN-RNN)
   MDN-RNN epoch  10/60: z NLL  0.739  reward MSE 0.0071
   MDN-RNN epoch  20/60: z NLL  0.651  reward MSE 0.0039
   MDN-RNN epoch  30/60: z NLL  0.627  reward MSE 0.0029
   MDN-RNN epoch  40/60: z NLL  0.616  reward MSE 0.0026
   MDN-RNN epoch  50/60: z NLL  0.609  reward MSE 0.0023
   MDN-RNN epoch  60/60: z NLL  0.604  reward MSE 0.0021
4) Evolving C inside the dream (CMA-ES)
   generation  10: dream return mean 2.36  best 3.05
   generation  20: dream return mean 3.19  best 3.70
   generation  30: dream return mean 3.25  best 3.72
   generation  40: dream return mean 3.38  best 3.72
   generation  50: dream return mean 3.46  best 3.89
   generation  60: dream return mean 3.48  best 4.02
   generation  70: dream return mean 3.49  best 3.97
   generation  80: dream return mean 3.59  best 4.07
5) Testing in the real environment
   random policy                 1.37 +/- 1.01 balls caught (out of 5)
   world model + dream-trained C  5.00 +/- 0.00 balls caught (out of 5)
   world model + CEM planning     4.70 +/- 0.56 balls caught (out of 5)
   hand-coded policy              5.00 +/- 0.00 balls caught (out of 5)
Finished in 12.4 minutes. Models and logs saved to 'runs/'.
```

Table 3. Balls caught per episode in the real game (out of 5; mean ± standard deviation). All policies face the same sequence of balls. †20 episodes, because planning is slow; all others 100 episodes. (Over 2,000 episodes the random policy averages 1.6; 100 episodes give a noisy estimate.)

Policy	How it chooses actions	Balls caught
Random	random actions, each held for 1–8 steps	1.37 ± 1.01
World model + C	linear controller trained *only* in the dream	$\mathbf{5.00 \pm 0.00}$
World model + planning	CEM: imagines 100×16 steps, twice, before every real step	$4.70 \pm 0.56^{\dagger}$
Hand-coded	locates ball and paddle by colour, moves towards the ball	5.00 ± 0.00

The controller never saw the real game during training, yet it catches every ball in all 100 test episodes (Table 3), matching the hand-coded policy. Two details are worth noticing.

- **The dream is harder than reality.** The final controller scores about 3.7 in the dream (Figure 4d) but 5.0 in the real game. M's predicted reward at a catch is often a little uncertain, and its sampled futures are noisier than the real game; we also sample at $\tau = 1.15$. The paper saw the same pattern in Doom (918 in the dream, 1,092 in reality).
- **Planning works without any policy training, but it is slow.** The planner imagines $100 \times 16 \times 2 = 3{,}200$ steps for every real step. Testing it took about 2.6 minutes for 20 episodes, compared with about 20 seconds for 100 episodes with C, which needs a single matrix multiplication per step. This is the general trade-off between learning a policy and planning at decision time.

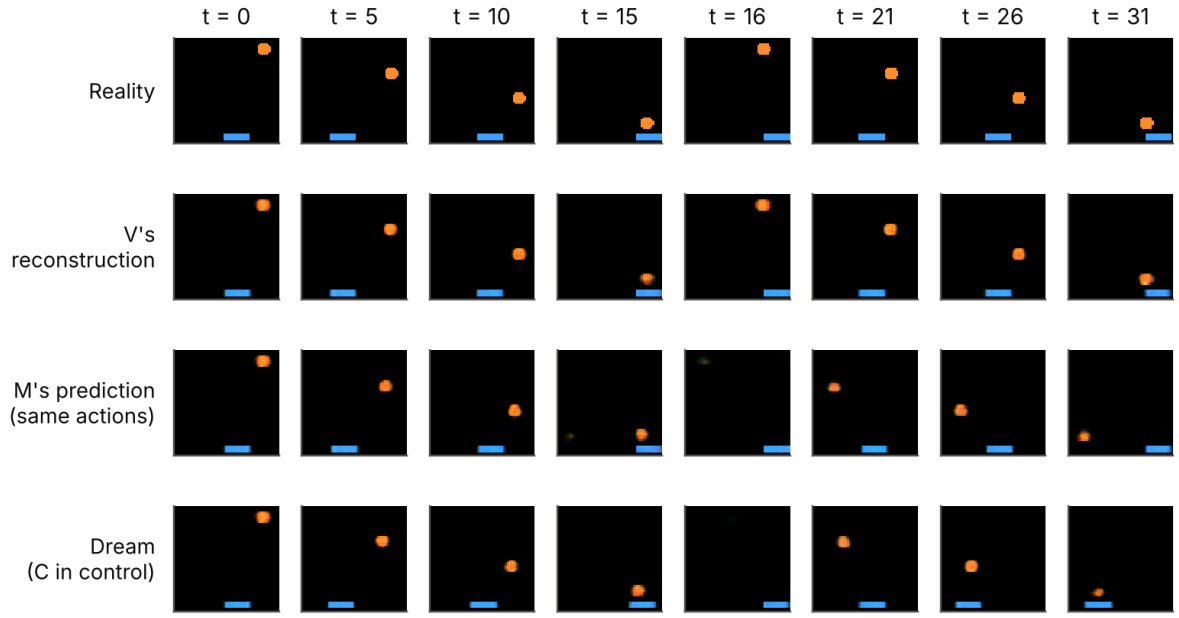


Figure 3. What the agent sees and imagines. **Row 1:** the real game played by the trained controller; it catches the first ball at $t = 15$, and a new ball appears at $t = 16$. **Row 2:** V's reconstructions of the same frames, each made from just 32 numbers. **Row 3:** M's *open-loop* prediction, given only the first frame and the same actions. It is accurate until the first landing; then M imagines the new ball at a different, randomly sampled place, a plausible future that simply did not happen. Right after a landing M is unsure where the next ball will be, so the new ball "materialises" over a few steps. **Row 4:** a dream in which C controls the paddle; it chases the dreamed ball just as it would in reality.

5.10 Experiments: when does dream training transfer to reality?

`dream_experiments.py` (appendix) reuses V and evolves new controllers inside different world models: (A) weaker versions of M, trained on less data or for fewer epochs, and (B) the full M sampled at different temperatures. Each controller is scored in the dream (500 dreams) and in the real game (100 episodes).

Table 4. Dream score versus real score (balls caught out of 5). Same procedure for every row: CMA-ES for 80 generations inside the given world model. The random policy scores 1.37 in reality.

	World model M	τ	Score in the dream	Score in reality
A	trained on 20 episodes, 60 epochs	1.15	5.65	1.12 ± 0.87
	trained on 300 episodes, 3 epochs	1.15	5.02	1.12 ± 0.87
	trained on 300 episodes, 60 epochs (full)	1.15	3.71	5.00 ± 0.00
B	full	0.10	4.06	4.94 ± 0.24
	full	0.50	3.74	5.00 ± 0.00
	full	1.00	3.76	5.00 ± 0.00
	full	1.15	3.71	5.00 ± 0.00
	full	1.50	3.59	5.00 ± 0.00

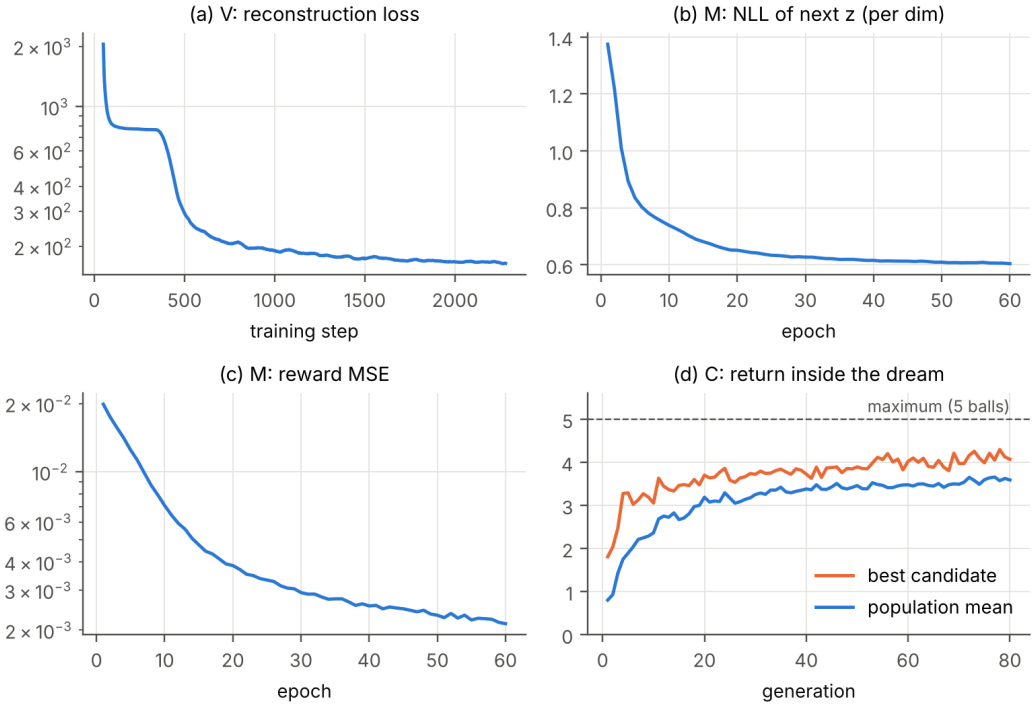


Figure 4. Training curves. (a) V first learns to output an empty black frame (the plateau), then learns to draw the paddle and the ball. (b,c) M's prediction losses on the training data. (d) The dream return of CMA-ES's candidates. It stays below 5 because the dream is harder than the real game (see text).

A higher dream score does not mean a better agent

With the weak world models (rows A1 and A2), the controllers achieve their *highest* dream scores, up to 5.65, which is more than the 5 balls that are physically possible. In reality they catch fewer balls than the random policy. When we watched one of them, it pushed the paddle against the right wall for 85–93% of each episode: the weak model had wrongly learned that this earns rewards, and evolution found and exploited that error. This is the "cheating the world model" effect described by Ha and Schmidhuber. *Always* evaluate in the real environment, and treat the dream score only as a training signal.

Part B gives a different answer from the paper's Doom experiment. In our game the controller transfers at every temperature, even $\tau = 0.1$. The only random event here, where the next ball appears, is easy to model, and the controller cannot "switch it off" the way Doom's agent made the dreamed monsters stop shooting. Temperature is a safety knob whose importance depends on the environment; the accuracy of the world model (part A) mattered far more here.

6 From this toy to today's world models

The V–M–C recipe contains the essential ideas, and modern systems keep them while changing how each part is built and trained. Table 6 gives an overview.

6.1 Dreamer: learning the model and the behaviour together

PlaNet [15] introduced the *recurrent state-space model* (RSSM), and the Dreamer family [16, 17, 18] learns behaviour inside it. Table 5 maps our components to Dreamer's. DreamerV3, published in *Nature* in 2025, outperforms specialised methods on over 150 tasks with a single set of hyperparameters, and was the first algorithm to collect diamonds in Minecraft from scratch,

without human data [18]. Dreamer 4 trains its agent by reinforcement learning inside a fast Transformer world model and obtains Minecraft diamonds purely from offline data, without any environment interaction [19].

Table 5. Our V–M–C agent compared with the Dreamer family.

This chapter	Dreamer
V and M trained separately	one model trained end to end (the RSSM)
LSTM memory h_t plus an MDN over the next code	recurrent state h_t plus a stochastic state z_t (Gaussian in v1, categorical in v2 and v3), with a *posterior* that sees the frame and a *prior* that must predict without it
losses: reconstruction + KL to $\mathcal{N}(0, I)$; NLL of the next code; reward MSE	reconstruction + reward (+ episode continuation in v2 and v3) + KL between posterior and prior
linear C evolved with CMA-ES on full-episode dreams	neural actor and critic trained on short imagined trajectories
one round of random data	a continual loop: act, store data, train the model, train the actor-critic

6.2 Planning with learned models

MuZero [31] learns a model that predicts only what planning needs, namely rewards, values and action probabilities, without ever reconstructing observations, and plans with Monte-Carlo tree search. It mastered Atari, Go, chess and shogi. TD-MPC2 [22] plans in the latent space of a decoder-free world model. It performed strongly on 104 continuous-control tasks with one set of hyperparameters and scaled to a single 317M-parameter agent for 80 tasks.

6.3 Generative world models: frames, tokens and video

Instead of a small latent code, some world models generate observations directly. IRIS [26] turns frames into discrete tokens and predicts them with a Transformer. DIAMOND [1] uses a diffusion model as the world model, which preserves visual details, and its authors also released a playable world model of the game *Counter-Strike*. At a much larger scale, Genie [4], an 11-billion-parameter model trained on unlabelled Internet videos, generates action-controllable virtual worlds by learning its own *latent actions* from video alone. Genie 3 generates interactive worlds that can be navigated in real time at 720p and 24 frames per second and stay consistent for a few minutes [7]. NVIDIA's Cosmos provides open *world foundation models* that can be fine-tuned for robotics and autonomous driving [28].

6.4 Predicting in representation space: JEPA

LeCun argues that predicting every pixel is wasteful and that a world model should predict *abstract representations* of the future, using a *joint-embedding predictive architecture* (JEPA) [25]. V-JEPA 2 [2] is pretrained on over a million hours of Internet video. Its action-conditioned version, V-JEPA 2-AC, is then post-trained on less than 62 hours of robot videos and controls real robot arms zero-shot. To pick and place objects, it uses the cross-entropy method to choose actions whose predicted representation is closest to that of a goal image. This is the same loop as our `cem_plan`, with "distance to the goal" in place of "predicted reward".

Table 6. A short map of influential world-model methods.

Method (year)	What the model predicts	How behaviour is obtained
Dyna (1991) [32]	next state and reward	RL on real plus simulated experience
PILCO (2011) [8]	next state, with uncertainty (Gaussian process)	policy search through the model
World Models (2018) [13]	next VAE code (MDN-RNN), episode end	evolution strategy, even fully in the dream
PlaNet (2019) [15]	latent states (RSSM), rewards	online planning with CEM
Dreamer v1–v3 (2020–25) [16, 17, 18]	latent states, rewards, continuation	actor-critic in imagination
MuZero (2020) [31]	rewards, values, policies (no pixels)	Monte-Carlo tree search
IRIS (2023) [26]	image tokens (Transformer)	actor-critic in imagination
DIAMOND (2024) [1]	next frame (diffusion model)	actor-critic in imagination
TD-MPC2 (2024) [22]	latent states, rewards, values (no decoder)	planning (MPPI) with a learned policy prior
Genie (2024), Genie 3 (2025) [4, 7]	video frames given (latent) actions	a generator of interactive worlds
Cosmos (2025) [28]	future video (diffusion and autoregressive)	foundation model, fine-tuned per application
V-JEPA 2-AC (2025) [2]	future representations (no pixels)	CEM planning towards a goal image
Dreamer 4 (2025) [19]	future video, with a fast Transformer	RL in imagination from offline data

7 Practical lessons

- **Never trust the dream score.** Always test in the real environment (Table 4).
- **Expect exploitation.** An optimiser will find the model's mistakes. Common remedies are more and more varied data, stochastic models with a sensible temperature, ensembles of models that expose uncertainty, short imagined horizons, and continual training on fresh real data.
- **Errors compound.** A small one-step error grows over many imagined steps (row 3 of Figure 3). Many methods therefore imagine only short futures and use a learned value function to account for what lies beyond.
- **Data coverage limits the model.** A model cannot know what it has never seen. If random actions do not reach the important states, collect data with the improving agent and retrain (iterative training).
- **Decide what to predict.** A reconstruction loss spends capacity on every pixel, relevant or not. Ha and Schmidhuber's VAE reproduced irrelevant brick patterns on Doom's walls but missed task-relevant road tiles in car racing. Reward- and value-focused models (MuZero, TD-MPC2) and JEPA avoid pixels altogether.
- **Check each part separately.** Look at V's reconstructions, compare M's predictions with reality, and compare the agent with simple baselines before you trust the whole system.

8 Exercises

1. **Is memory needed?** Give C only z_t (replace h by zeros in `controller_action`) and compare with the paper's car-racing result.

2. **Make memory necessary.** Give the ball a random horizontal speed that bounces off the walls. Now a single frame no longer tells where the ball will land. Does C still work with and without h_t?
3. **Catch a cheater.** Train M on only 20 episodes and evolve C in its dream. Then adapt `make_figures.py` to watch what C does in the dream and in reality.
4. **Iterate.** Collect 100 episodes with the trained C, add them to the data, retrain M and C, and check whether the dream score moves closer to the real score.
5. **Use gradients instead of evolution.** Replace CMA-ES with a neural policy trained by back-propagating the dream return through M, using the reparameterised Gaussian sample. This is the route Dreamer takes.
6. **A harder world.** Plug in Gymnasium's *CarRacing* (resize frames to 64×64; three action dimensions) and compare with the paper.

9 Where to learn more

- **Start with the original.** The interactive article *World Models* [13] (`https://worldmodels.github.io`) has in-browser demos, and the authors' code is at `https://github.com/hardmaru/WorldModelsExperiments`.
- **Books.** Sutton and Barto [33], Chapter 8, covers planning and learning with models (Dyna). Foster [10], Chapter 12, is a step-by-step walkthrough of the V–M–C model in TensorFlow/Keras.
- **Courses.** UC Berkeley CS285, Deep Reinforcement Learning (Sergey Levine), has lectures on model-based RL and on learning policies with models (`https://rail.eecs.berkeley.edu/deeprlcourse/`). Datawhale's free *Learn World Models* course (in English and Chinese; an alpha preview in 2026) has PyTorch projects that go from a VAE to an RSSM, a Dreamer agent and a Transformer dynamics model [6].
- **Code to read next.** danijar/dreamerv3 (official, JAX); NM512/dreamerv3-torch (PyTorch); eloialonso/iris; eloialonso/diamond; nicklashansen/tdmpc2; facebookresearch/vjepa2.
- **Surveys and tutorials.** Moerland et al. [27] on model-based RL; Ding et al. [9] on world models in general; Oh et al. [29], a tutorial that builds up from Ha–Schmidhuber to Dreamer, MuZero, Genie and JEPA; and Zhang et al. [34] on world models for robotics. Curated, frequently updated paper lists include Awesome-World-Models.

References

[1] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, F. Fleuret. Diffusion for world modeling: visual details matter in Atari. *NeurIPS*, 2024.

[2] M. Assran et al. V-JEPA 2: self-supervised video models enable understanding, prediction and planning. arXiv:2506.09985, 2025.

[3] C. M. Bishop. Mixture density networks. Technical report NCRG/94/004, Aston University, 1994.

[4] J. Bruce et al. Genie: generative interactive environments. *ICML*, 2024.

[5] K. J. W. Craik. *The Nature of Explanation*. Cambridge University Press, 1943.

[6] Datawhale. Learn World Models (course, alpha preview). `https://datawhalechina.github.io/learn-world-model/`, 2026.

[7] Google DeepMind. Genie 3: a new frontier for world models. Blog post, 5 August 2025.

[8] M. P. Deisenroth, C. E. Rasmussen. PILCO: a model-based and data-efficient approach to policy search. *ICML*, 2011.

[9] J. Ding et al. Understanding world or predicting future? A comprehensive survey of world models. *ACM Computing Surveys*, 2025. arXiv:2411.14499.

[10] D. Foster. *Generative Deep Learning*, 2nd edition, Chapter 12: World Models. O'Reilly, 2023.

[11] A. Graves. Generating sequences with recurrent neural networks. arXiv:1308.0850, 2013.

[12] D. Ha, D. Eck. A neural representation of sketch drawings. *ICLR*, 2018.

[13] D. Ha, J. Schmidhuber. World models. arXiv:1803.10122, 2018. Interactive version: `https://worldmodels.github.io`.

[14] D. Ha, J. Schmidhuber. Recurrent world models facilitate policy evolution. *NeurIPS*, 2018.

[15] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, J. Davidson. Learning latent dynamics for planning from pixels. *ICML*, 2019.

[16] D. Hafner, T. Lillicrap, J. Ba, M. Norouzi. Dream to control: learning behaviors by latent imagination. *ICLR*, 2020.

[17] D. Hafner, T. Lillicrap, M. Norouzi, J. Ba. Mastering Atari with discrete world models. *ICLR*, 2021.

[18] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. arXiv:2301.04104.

[19] D. Hafner, W. Yan, T. Lillicrap. Training agents inside of scalable world models. arXiv:2509.24527, 2025.

[20] N. Hansen, A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 2001.

[21] N. Hansen. The CMA evolution strategy: a tutorial. arXiv:1604.00772, 2016.

[22] N. Hansen, H. Su, X. Wang. TD-MPC2: scalable, robust world models for continuous control. *ICLR*, 2024.

[23] S. Hochreiter, J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8), 1997.

[24] D. P. Kingma, M. Welling. Auto-encoding variational Bayes. *ICLR*, 2014.

[25] Y. LeCun. A path towards autonomous machine intelligence. OpenReview, version 0.9.2, 2022.

[26] V. Micheli, E. Alonso, F. Fleuret. Transformers are sample-efficient world models. *ICLR*, 2023.

[27] T. M. Moerland, J. Broekens, A. Plaat, C. M. Jonker. Model-based reinforcement learning: a survey. *Foundations and Trends in Machine Learning*, 16(1), 2023.

[28] NVIDIA. Cosmos world foundation model platform for physical AI. arXiv:2501.03575, 2025.

[29] I.-S. Oh et al. A tutorial on world models and physical AI. *ACM Computing Surveys*, 2026. arXiv:2606.12783.

[30] R. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2), 1999.

[31] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588, 2020.

[32] R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4), 1991.

[33] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[34] X. Zhang et al. From world models to world action models: a concise tutorial for robotics. arXiv:2607.00836, 2026.

A Appendix: the two helper scripts

`dream_experiments.py` produces Table 4, and `make_figures.py` produces Figures 3 and 4. Both load the models that `world_model.py` saved in `runs/`.

Listing 8 `dream_experiments.py`

```
"""
dream_experiments.py: when does a controller trained in the dream work in reality?
Reuses V and M saved by world_model.py and runs two small experiments:
  A. world-model quality: evolve C inside weaker world models (less data / less training)
  B. dream temperature: evolve C inside the full world model at several temperatures
Each controller is scored inside the dream (500 dreams) and in the real game
(100 episodes).

Run after world_model.py:  python dream_experiments.py   (about 12 minutes on 2 CPU cores)
"""
import json
from dataclasses import replace

import torch

import world_model as wm

cfg = wm.Config(**json.load(open("runs/log.json"))["config"])
vae = wm.ConvVAE(cfg.z_dim).to(wm.DEVICE)
vae.load_state_dict(torch.load("runs/vae.pt", map_location=wm.DEVICE))
full_m = wm.MDNRNN(cfg.z_dim, 1, cfg.hidden, cfg.n_mix).to(wm.DEVICE)
full_m.load_state_dict(torch.load("runs/mdnrnn.pt", map_location=wm.DEVICE))

# The same random-policy data as in world_model.py (same seed), encoded by the saved V
obs, act, rew = wm.collect_episodes(cfg.n_episodes, cfg.seed)
mu, logvar = wm.encode_episodes(vae, obs)
act, rew = torch.from_numpy(act), torch.from_numpy(rew)


def dream_vs_reality(rnn, tau):
    """Evolve C inside `rnn` at temperature tau; return its dream and real scores."""
    torch.manual_seed(cfg.seed)
    run_cfg = replace(cfg, temperature=tau)
    params, _ = wm.train_controller_in_dream(rnn, mu[:, 0], logvar[:, 0], run_cfg)
    run_cfg.dream_rollouts = 500
    dream = wm.dream_returns(rnn, params.cpu().numpy(), mu[:, 0], logvar[:, 0], run_cfg)
    dream = float(dream[0])
    real, real_std = wm.evaluate(wm.WorldModelAgent(vae, rnn, params), cfg.eval_episodes)
    return {"dream": dream, "real": real, "real_std": real_std}


results = {"A": [], "B": []}
print("A. World-model quality (temperature 1.15)")
for n_episodes, epochs in [(20, 60), (300, 3), (300, 60)]:
    if (n_episodes, epochs) == (cfg.n_episodes, cfg.rnn_epochs):
        rnn = full_m                            # the model trained by world_model.py
    else:
        torch.manual_seed(cfg.seed)
        weak_cfg = replace(cfg, rnn_epochs=epochs)
        rnn, _ = wm.train_mdnrnn(mu[:n_episodes], logvar[:n_episodes], act[:n_episodes],
                                 rew[:n_episodes], weak_cfg)
    r = {"episodes": n_episodes, "epochs": epochs,
         **dream_vs_reality(rnn, cfg.temperature)}
    results["A"].append(r)
    print(f"   M from {n_episodes:3d} episodes, {epochs:2d} epochs: "
```

```
          f"dream {r['dream']:.2f}, "
          f"reality {r['real']:.2f} +/- {r['real_std']:.2f}")

print("B. Dream temperature (full world model)")
for tau in [0.1, 0.5, 1.0, 1.15, 1.5]:
    r = results["A"][-1] if tau == cfg.temperature else dream_vs_reality(full_m, tau)
    results["B"].append({"temperature": tau, "dream": r["dream"], "real": r["real"],
                         "real_std": r["real_std"]})
    print(f"   tau = {tau:4.2f}: dream {r['dream']:.2f}, "
          f"reality {r['real']:.2f} +/- {r['real_std']:.2f}")

with open("runs/dream_experiments.json", "w") as f:
    json.dump(results, f, indent=1)
```

Listing 9 `make_figures.py`

```
"""
make_figures.py: draws the chapter's figures from the files saved by world_model.py.
Run after world_model.py:  python make_figures.py
"""
import json

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch

import world_model as wm

BLUE, ORANGE, INK, MUTED, GRID = "#2a78d6", "#eb6834", "#0b0b0b", "#52514e", "#e4e3df"
plt.rcParams.update({"font.size": 9, "axes.edgecolor": MUTED, "axes.labelcolor": INK,
                     "axes.titlesize": 9.5, "xtick.color": MUTED, "ytick.color": MUTED,
                     "axes.spines.top": False, "axes.spines.right": False,
                     "axes.grid": True,
                     "grid.color": GRID, "grid.linewidth": 0.6, "lines.linewidth": 1.6})

log = json.load(open("runs/log.json"))
cfg = wm.Config(**log["config"])
vae = wm.ConvVAE(cfg.z_dim).to(wm.DEVICE)
vae.load_state_dict(torch.load("runs/vae.pt", map_location=wm.DEVICE))
rnn = wm.MDNRNN(cfg.z_dim, 1, cfg.hidden, cfg.n_mix).to(wm.DEVICE)
rnn.load_state_dict(torch.load("runs/mdnrnn.pt", map_location=wm.DEVICE))
params = torch.load("runs/controller.pt", map_location=wm.DEVICE)
torch.manual_seed(3)

# ---- Figure: training curves of V, M and C ------------------------------------------
smooth = lambda y, k: np.convolve(y, np.ones(k) / k, mode="valid")
fig, ax = plt.subplots(2, 2, figsize=(7.2, 5.0))
ax = ax.ravel()
rec = smooth(np.array(log["vae"])[:, 0], 50)
ax[0].plot(np.arange(50, 50 + len(rec)), rec, color=BLUE)
ax[0].set(title="(a) V: reconstruction loss", xlabel="training step", yscale="log")
m = np.array(log["mdnrnn"])
ax[1].plot(np.arange(1, len(m) + 1), m[:, 0], color=BLUE)
ax[1].set(title="(b) M: NLL of next z (per dim)", xlabel="epoch")
ax[2].plot(np.arange(1, len(m) + 1), m[:, 1], color=BLUE)
ax[2].set(title="(c) M: reward MSE", xlabel="epoch", yscale="log")
es = np.array(log["cmaes"])
gens = np.arange(1, len(es) + 1)
ax[3].plot(gens, es[:, 1], color=ORANGE, label="best candidate")
```

```python
ax[3].plot(gens, es[:, 0], color=BLUE, label="population mean")
ax[3].axhline(5, color=MUTED, lw=0.8, ls="--")
ax[3].text(len(es), 5.07, "maximum (5 balls)", ha="right", va="bottom", color=MUTED,
           fontsize=8)
ax[3].set(title="(d) C: return inside the dream", xlabel="generation", ylim=(0, 5.6))
ax[3].legend(frameon=False, loc="lower right")
fig.tight_layout()
fig.savefig("fig_training.png", dpi=220)

# ---- Figure: reality vs. what V, M and C "see" ---------------------------------------
T = 32
env, agent = wm.CatchEnv(7), wm.WorldModelAgent(vae, rnn, params)
agent.reset()
frames, actions = [env.reset()], []
for t in range(T):                                      # C plays in the real environment
    actions.append(agent(frames[-1]))
    frames.append(env.step(actions[-1])[0])

with torch.no_grad():
    mu, _ = vae.encode(wm.to_tensor(np.stack(frames)))
    recon = vae.decode(mu)                              # what the agent "sees" through V
    z, state, predicted = mu[:1], None, [vae.decode(mu[:1])]
    for t in range(T):                              # M imagines the result of the same actions
        a = torch.tensor([[[actions[t]]]], device=wm.DEVICE)
        logpi, mmu, logsigma, _, state = rnn(z[:, None], a, state)
        z = wm.sample_mdn(logpi[:, 0], mmu[:, 0], logsigma[:, 0])
        predicted.append(vae.decode(z))
    z, state, dream = mu[:1], None, [vae.decode(mu[:1])]
    h = torch.zeros(1, cfg.hidden, device=wm.DEVICE)
    for t in range(T):                              # C plays inside the dream (closed loop)
        a = wm.controller_action(params, z[None], h[None])[0]
        logpi, mmu, logsigma, _, state = rnn(z[:, None], a[:, None], state)
        z, h = wm.sample_mdn(logpi[:, 0], mmu[:, 0], logsigma[:, 0]), state[0][0]
        dream.append(vae.decode(z))

to_numpy = lambda x: x.permute(0, 2, 3, 1).clamp(0, 1).cpu().numpy()   # [N, 64, 64, 3]
rows = [("Reality", np.stack(frames) / 255.0),
        ("V's\nreconstruction", to_numpy(recon)),
        ("M's prediction\n(same actions)", to_numpy(torch.cat(predicted))),
        ("Dream\n(C in control)", to_numpy(torch.cat(dream)))]
times = [0, 5, 10, 15, 16, 21, 26, 31]
fig, ax = plt.subplots(len(rows), len(times), figsize=(7.2, 4.1))
for r, (label, images) in enumerate(rows):
    for c, t in enumerate(times):
        ax[r, c].imshow(images[t], interpolation="nearest")
        ax[r, c].set_xticks([])
        ax[r, c].set_yticks([])
        ax[r, c].grid(False)
        if r == 0:
            ax[r, c].set_title(f"t = {t}", color=INK)
    ax[r, 0].set_ylabel(label, rotation=0, ha="right", va="center", color=INK, labelpad=8)
fig.tight_layout()
fig.savefig("fig_dream.png", dpi=220)
```